

Assignment 02

IP Addressing, Routing Algorithms & Congestion Control

Name: Deven

UID: 24BCF10060

Subject: Computer Networks

Q1. IP Addresses, Classes, Subnetting and Supernetting

What is an IP Address?

An IP address is a 32-bit (in IPv4) numeric identifier assigned to every device on a network. It uniquely identifies the device and the network it belongs to, and consists of two parts: a Network ID (identifying the network) and a Host ID (identifying the specific device within that network).

Why Do We Need Classes in IPv4?

When IPv4 was designed, networks of very different sizes had to be accommodated — from a handful of hosts in a small office to millions of hosts on a large backbone. Classful addressing solved this by dividing the address space into classes, each reserving a different number of bits for the Network ID and the Host ID, so that organisations could be given an address block matching the size of their network instead of wasting addresses on a one-size-fits-all scheme.

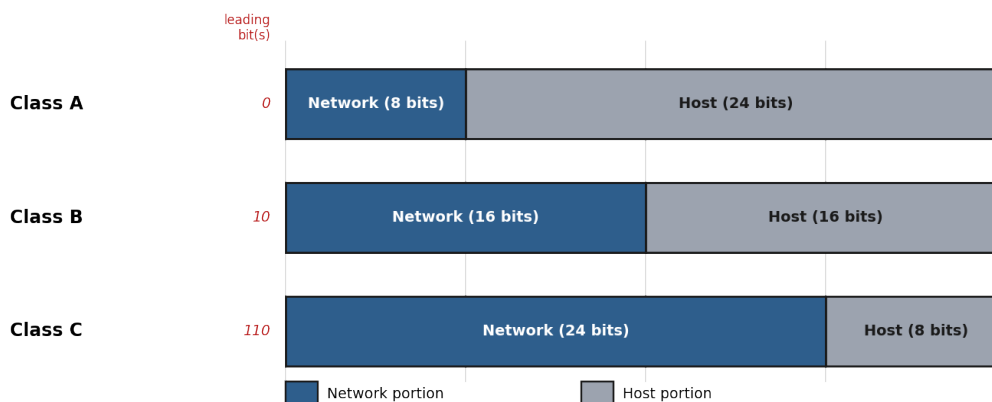


Fig. 1 — Bit allocation between the Network and Host portions for Class A, B and C

The Five IPv4 Classes

Class	First Octet Range	Default Mask	No. of Networks	Hosts / Network
A	1 – 126	255.0.0.0 (/8)	126	16,777,214
B	128 – 191	255.255.0.0 (/16)	16,384	65,534
C	192 – 223	255.255.255.0 (/24)	2,097,152	254
D	224 – 239	— (Multicast)	—	—
E	240 – 255	— (Experimental / Reserved)	—	—

Class A is meant for a very small number of very large networks (few Network bits, many Host bits); Class C is the opposite — a very large number of small networks. Class D addresses are not divided into Network/Host portions at all and are reserved for multicast groups, while Class E is reserved for experimental and future use.

Subnetting

Subnetting is the process of borrowing bits from the Host portion of an address to create several smaller sub-networks out of one larger network. It is used to avoid wasting addresses, to improve routing efficiency, and to organise a large network into manageable, isolated segments.

Example: The network 192.168.1.0 /24 (254 usable hosts) can be subnetted by borrowing 2 host bits, giving a /26 mask (255.255.255.192). This produces 4 subnets, each with 62 usable hosts: 192.168.1.0/26, 192.168.1.64/26, 192.168.1.128/26 and 192.168.1.192/26.

Supernetting

Supernetting is the reverse idea — combining several contiguous smaller networks into one larger block by borrowing bits back from the Network portion into the Host portion (i.e. shortening the prefix). It is mainly used for route summarisation, to keep routing tables small.

Example: The four Class C networks 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24 can be advertised by a router as a single supernet, 192.168.0.0/22, instead of four separate routing table entries.

Q2. Link State vs Distance Vector Routing

Both are used by routers to build routing tables and find the least-cost path to every destination, but they differ fundamentally in how much of the network each router knows and how that knowledge is obtained.

Aspect	Distance Vector	Link State
Knowledge of topology	Only distance and direction to each destination, learnt from neighbours	Complete map of the entire network topology
Information exchanged	Its entire routing table, sent to directly connected neighbours only	Link-State Advertisements (LSAs), flooded to every router in the area
Underlying algorithm	Bellman-Ford (distributed)	Dijkstra's shortest-path algorithm (run locally)
Update trigger	Periodically, at fixed intervals	Only when a link/topology change occurs
Convergence speed	Slower, prone to routing loops	Faster and more accurate
Example protocol	RIP	OSPF

Router link costs used to illustrate shortest-path computation (A → D)

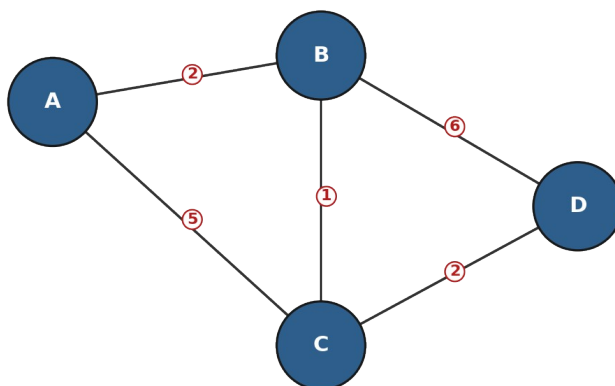


Fig. 2 — A small network with link costs, used below for both algorithms

Distance Vector Example (Bellman-Ford)

Using the graph above, router A initially only knows its direct link costs: B = 2 and C = 5. After exchanging distance vectors with its neighbours, A learns from B that B can reach C in 1 hop and D in 6 hops. A then applies the Bellman-Ford relaxation $DA(y) = \min \text{ over neighbour } v \text{ of } [\text{cost}(A,v) + Dv(y)]$: the cost to C improves to $2 + 1 = 3$ (via B, better than the direct 5), and the cost to D becomes $3 + 2 = 5$ (A → B → C → D), which is cheaper than the direct A → B → D path of $2 + 6 = 8$. This iterative, neighbour-by-neighbour update is repeated until every router's table stops changing.

Link State Example (Dijkstra)

With link-state routing, every router first floods the costs of all its links to every other router, so that A ends up with the complete graph shown in Fig. 2. A then runs Dijkstra's algorithm locally: starting from A (cost 0), it repeatedly picks the nearest unvisited node and relaxes its neighbours — B is reached at cost 2, then C at cost 3 (via B), and finally D at cost 5 (via B – C). This gives the same least-cost path, A – B – C – D with total cost 5, but computed directly from full topology knowledge rather than through repeated rounds of neighbour gossip.

Q3. Congestion Control and Fragmentation

What is Congestion?

Congestion occurs when the number of packets arriving at a router or being carried by a network exceeds the capacity of that router or link to handle them. Queues start to build up and overflow, causing increased delay, dropped packets, and unnecessary retransmissions, which in turn adds even more traffic to an already overloaded network — a condition that can spiral into congestive collapse if left unchecked.

Why Do We Need to Control Congestion?

Congestion control is necessary to keep network throughput high, keep end-to-end delay and packet loss within acceptable limits, ensure that bandwidth is shared fairly between competing flows, and prevent the network from collapsing under its own retransmission traffic.

Leaky Bucket vs Token Bucket

Both are traffic-shaping mechanisms that regulate how bursty traffic is allowed to enter the network, but they take opposite approaches to bursts.

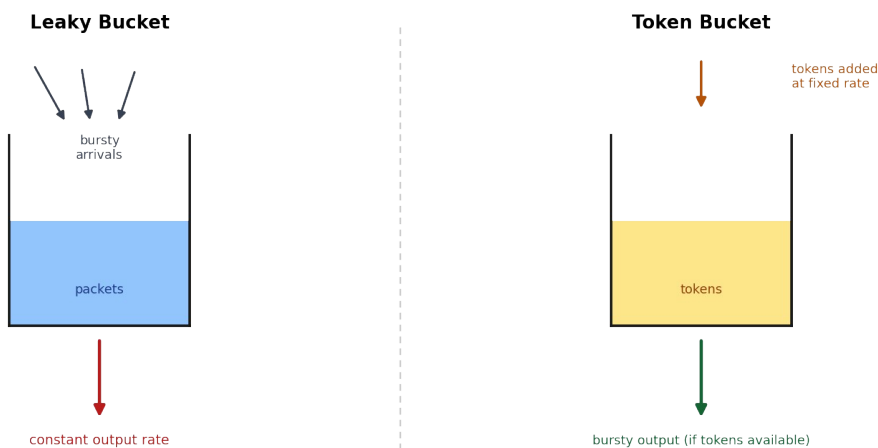


Fig. 3 — Leaky bucket enforces a fixed output rate; token bucket allows saved-up bursts

Leaky Bucket	Token Bucket
Output rate is always constant, regardless of how bursty the input is.	Output can be bursty, as long as enough tokens have accumulated.
Excess packets that don't fit in the bucket are simply discarded.	Tokens accumulate (up to a limit) during idle periods and can later be spent on a burst.
Cannot take advantage of idle time to send a later burst.	Rewards idle periods by allowing a burst afterwards.
Analogy: water leaking from a fixed hole at a steady rate.	Analogy: a packet can leave only if it can 'pay' with an available token.

Example: with a leaky bucket of output rate 2 Mbps, even if 10 Mbps of traffic arrives for one second, only 2 Mbps leaves every second and the rest is queued or dropped. With a token bucket that also generates tokens at 2 Mbps but can store

up to 5 Mb worth of tokens, a sender that was idle for a while can burst at a much higher rate for a short time until its saved-up tokens run out.

Why Do We Need Fragmentation?

Different physical networks support different Maximum Transmission Units (MTUs) — for example, Ethernet typically allows 1500 bytes per frame, while some other links allow less. When an IP datagram is larger than the MTU of the outgoing link, it cannot be transmitted as a single unit, so the router breaks it into smaller pieces called fragments, each small enough to fit the link's MTU. The IP header's Identification, Flags (DF/MF) and Fragment Offset fields let the destination host reassemble the original datagram correctly, even if the fragments arrive out of order.